

Национальный исследовательский университет ИТМО

Факультет безопасности информационных технологий

Алгоритмы и структуры данных

Лабораторная работа №4

**«Цифровая сортировка стека на базе массива с
поиском методом Бойера-Мура и записью результата
в красно-чёрное дерево»**

Выполнил:

Студент группы N3247

Сивоконь А.А.

(подпись)

Преподаватель:

Ерофеев С.А.

(отметка о выполнении)

(подпись)

Санкт-Петербург

2026

Содержание

1	Постановка задачи	2
2	Описание структур данных и алгоритмов	3
2.1	Стек на базе динамического массива	3
2.2	Алгоритм цифровой сортировки LSD через стеки-корзины .	3
2.3	Поиск методом Бойера-Мура	4
2.4	Красно-чёрное дерево	5
2.5	Маршрут программы	5
3	Описание используемых переменных	6
4	Тестирование программы	9
4.1	Тест 1: стандартная сортировка и поиск	9
4.2	Тест 2: стек с отрицательными числами	9
4.3	Тест 3: стек из одного элемента	10
4.4	Тест 4: несуществующий файл	11
4.5	Тест 5: пустой файл	11
5	Блок-схемы алгоритма	12
6	Исходный код программы	13
6.1	Файл stack.h	13
6.2	Файл stack.c	15
6.3	Файл main.c	25
	Заключение	29
	Приложение А. Блок-схемы алгоритма	30

1. Постановка задачи

Цель работы: разработать программу цифровой сортировки (Radix Sort LSD) для последовательности целых чисел (включая отрицательные), считываемых из файла, используя в качестве основной и вспомогательной структуры данных **стек на базе динамического массива**; реализовать **поиск методом Бойера-Мура** в отсортированном стеке; записать результат сортировки в **красно-чёрное дерево**; определить сложность программы.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Реализовать стек на динамическом массиве с операциями `stack_init`, `stack_push`, `stack_pop`, `stack_peek`, `stack_is_empty`, `stack_free` (с автоматическим расширением);
2. Реализовать алгоритм цифровой сортировки LSD через 10 стековых корзин с учётом LIFO-особенностей (дополнительные реверсы для устойчивости);
3. Корректно обработать отрицательные числа: разделить стек на два вспомогательных, отсортировать каждый независимо, объединить результат с правильным порядком;
4. Реализовать поиск методом Бойера-Мура: сериализация отсортированного стека в строку с разделителями, поиск целыми токенами, возврат позиции числа (а не символа);
5. Реализовать красно-чёрное дерево и записать в него результат сортировки (с выводом inorder-обхода для проверки);
6. Оценить теоретическую сложность алгоритмов;
7. Провести тестирование программы на различных наборах данных;
8. Составить блок-схемы алгоритма и оформить отчёт.

2. Описание структур данных и алгоритмов

2.1. Стек на базе динамического массива

Основной структурой данных является стек (LIFO) на основе динамически выделяемого массива:

- Структура `Stack` содержит указатель `data` на буфер, индекс вершины `top` (-1 при пустом) и текущую ёмкость `capacity`.
- При переполнении ёмкость удваивается (начальная — 8), что обеспечивает амортизированно $O(1)$ на операцию `push`.
- Дно стека — `data[0]`, вершина — `data[top]`.

Операции стека:

Функция	Описание
<code>stack_init</code>	обнулить поля
<code>stack_is_empty</code>	вернуть <code>top == -1</code>
<code>stack_size</code>	вернуть количество элементов
<code>stack_push</code>	добавить на вершину (с расширением при необходимости)
<code>stack_pop</code>	извлечь с вершины
<code>stack_peek</code>	прочитать вершину без удаления
<code>stack_free</code>	освободить буфер

2.2. Алгоритм цифровой сортировки LSD через стеки-корзины

Цифровая сортировка — нерекурсивный алгоритм, обрабатывающий числа поразрядно от младшего разряда к старшему (LSD).

Особенность реализации: вместо очередей или деков используются **стеки-корзины** (10 штук). Стек — LIFO, поэтому для сохранения устойчивости и правильного порядка применяются дополнительные реверсы:

1. Для обработки элементов в порядке «от дна к вершине» (как в последовательности) исходный стек реверсируется во вспомогательный `rev`.

2. Распределение: элементы из `rev` извлекаются и добавляются в соответствующие корзины `buckets[digit]`.
3. Сбор: для каждой корзины 0..9 содержимое реверсируется через временный стек `temp` (двойной реверс восстанавливает порядок вставки в корзину), затем возвращается в основной стек.

Поддержка отрицательных чисел. Функция `radix_sort` разделяет исходный стек на `neg` (модули отрицательных) и `pos`. Обе части сортируются как неотрицательные. При сборке:

- Отрицательные извлекаются из `neg` напрямую (большие модули первыми → самые отрицательные оказываются внизу после `push`).
- Положительные реверсируются через вспомогательный стек, чтобы меньшие положительные добавлялись первыми.

После всех операций в стеке `data[0]` (дно) — наименьшее число, `data[top]` (вершина) — наибольшее.

2.3. Поиск методом Бойера-Мура

Поиск реализован для демонстрации классического алгоритма Бойера-Мура на результате сортировки.

Интеграция:

1. Из отсортированного стека строится текстовая строка вида `;val1;val2;...;valN`; (от дна к вершине).
2. Ключ преобразуется в паттерн `;ключ;`.
3. Выполняется Бойер-Мур (с таблицей плохого символа). Возвращается символьная позиция совпадения и количество просмотров (`views`).
4. Символьная позиция маппится в индекс числа в стеке (0 — у дна) с помощью накопления длин токенов.

Разделители гарантируют поиск целых чисел как токенов (например, `-1` не совпадёт внутри `-100`).

2.4. Красно-чёрное дерево

После сортировки результат записывается в красно-чёрное дерево (RBTree).

Реализована вставка с полной балансировкой (левый/правый повороты + `insert_fixup` по трём случаям). Дубликаты допускаются. Корень всегда чёрный.

Inorder-обход дерева должен выдавать ту же последовательность, что и содержимое стека от дна к вершине (дополнительная проверка корректности сортировки).

2.5. Маршрут программы

1. **Чтение входного файла** (`read_file`): считывание через `fscanf` + `stack_push`.
2. **Вывод исходного стека** (`stack_print`, от дна к вершине).
3. **Выполнение Radix Sort** (`radix_sort`).
4. **Вывод отсортированного стека**.
5. **Поиск методом Бойера-Мура** (`boyer_moore_search`): ключи из `argv` или `scanf`.
6. **Запись в красно-чёрное дерево** и вывод `inorder`.
7. **Освобождение памяти**.

3. Описание используемых переменных

В таблице 1 приведено описание переменных программы.

Таблица 1 – Описание переменных программы

Переменная	Тип	Границы	Назначение
Главная программа (main.c)			
filename	const char*	строка	Имя входного файла
s	Stack	структура	Основной стек программы
n	int	$[1; 2^{31} - 1]$	Количество прочитанных чисел
key	int	любое целое	Ключ поиска Бойера-Мура
pos	int	$[-1; n - 1]$	Позиция найденного элемента
views	int	$[0; n]$	Число просмотров при поиске
i	int	$[2; \text{argc} - 1]$	Индекс аргумента командной строки
Ввод из файла (main.c)			
fp	FILE*	адрес	Указатель на открытый файл
tmp	int	любое целое	Очередное считанное число
count	int	$[0; n]$	Количество прочитанных чисел
Стек (stack.c)			
s.data	int*	динамический	Буфер элементов стека
s.top	int	$[-1; n - 1]$	Индекс вершины стека

Переменная	Тип	Границы	Назначение
s.capacity	int	$[0; 2^{31} - 1]$	Текущая ёмкость буфера
new_cap	int	$\{8, 16, \dots\}$	Новая ёмкость при расширении
value	int	любое целое	Добавляемый/извлекаемый элемент
out	int*	адрес	Указатель на результат pop/peek
Цифровая сортировка (stack.c)			
buckets[10]	Stack[10]	структуры	Десять стеков-корзин
neg	Stack	структура	Стек модулей отрицательных чисел
pos	Stack	структура	Стек неотрицательных чисел
rev	Stack	структура	Вспомогательный реверс при раскладке
temp	Stack	структура	Вспомогательный реверс при сборе
pos_rev	Stack	структура	Реверс pos при финальной сборке
exp	int	$\{1, 10, \dots, 10^{d-1}\}$	Текущий множитель разряда
digit	int	$[0; 9]$	Цифра текущего разряда
max_val	int	$[0; 2^{31} - 1]$	Максимум для определения числа проходов

Переменная	Тип	Границы	Назначение
val	int	любое целое	Временная при перекладке элементов
Поиск Бойера-Мура (stack.c)			
text	char*	строка	Сериализованный стек вида ;val;val;
pat	char[40]	строка	Паттерн поиска вида ;ключ;
bad[256]	int[256]	$[-1; m - 1]$	Таблица плохого символа
shift	int	$[0; n - m]$	Смещение паттерна в тексте
j	int	$[-1; m - 1]$	Индекс в паттерне при сравнении
char_pos	int	$[-1; n - 1]$	Позиция совпадения в тексте
elem_idx	int	$[-1; n - 1]$	Индекс числа в стеке
Красно-чёрное дерево (stack.c)			
tree.root	RBNode*	адрес / NULL	Корень дерева
node.value	int	любое целое	Хранимое значение узла
node.color	Color	RED / BLACK	Цвет узла
node.left	RBNode*	адрес / NULL	Левый потомок
node.right	RBNode*	адрес / NULL	Правый потомок
node.parent	RBNode*	адрес / NULL	Родительский узел

4. Тестирование программы

4.1. Тест 1: стандартная сортировка и поиск

Файл `input.txt`: 170 45 75 90 802 24 2 66 -73 -4 123 -100 0 5
5 19 -19 300 -1

```
$ ./sort input.txt 75 -73 5

Чтение файла 'input.txt'...
Прочитано 19 чисел.

Исходный стек (от дна к вершине):
170 45 75 90 802 24 2 66 -73 -4 123 -100 0 5 5 19 -19 300 -1

Выполняем цифровую сортировку стека...
Отсортированный стек (от дна к вершине):
-100 -73 -19 -4 -1 0 2 5 5 19 24 45 66 75 90 123 170 300 802

Поиск методом Бойера-Мура
Ключ 75: НАЙДЕН на позиции 13 (просмотров: 24)
Ключ -73: НАЙДЕН на позиции 1 (просмотров: 6)
Ключ 5: НАЙДЕН на позиции 7 (просмотров: 16)

Записываем результат в красно-чёрное дерево...
Inorder обход дерева (должен совпадать с отсортированным стеком):
-100 -73 -19 -4 -1 0 2 5 5 19 24 45 66 75 90 123 170 300 802
```

Стек успешно отсортирован. Все три ключа найдены. Дубликат 5 присутствует на позиции 7. Inorder-обход красно-чёрного дерева совпадает с содержимым стека.

4.2. Тест 2: стек с отрицательными числами

Входной файл `neg.txt`: -15 3 -7 42 -2 18

```
$ ./sort neg.txt -15 5

Чтение файла 'neg.txt'...
Прочитано 6 чисел.

Исходный стек (от дна к вершине):
-15 3 -7 42 -2 18
```

```
Выполняем цифровую сортировку стека...
Отсортированный стек (от дна к вершине):
-15 -7 -2 3 18 42

Поиск методом Бойера-Мура
Ключ -15: НАЙДЕН на позиции 0 (просмотров: 5)
Ключ 5: не найден (просмотров: 6)

Записываем результат в красно-чёрное дерево...
Inorder обход дерева (должен совпадать с отсортированным стеком):
-15 -7 -2 3 18 42
```

Программа корректно разделила стек на $neg = \{15, 7, 2\}$ (по модулю) и $pos = \{3, 42, 18\}$, отсортировала каждый и собрала результат: сначала отрицательные по возрастанию $(-15, -7, -2)$, затем положительные $(3, 18, 42)$.

4.3. Тест 3: стек из одного элемента

```
$ ./sort one.txt 42 9

Чтение файла 'one.txt'...
Прочитано 1 число.

Исходный стек (от дна к вершине):
42

Выполняем цифровую сортировку стека...
Отсортированный стек (от дна к вершине):
42

Поиск методом Бойера-Мура
Ключ 42: НАЙДЕН на позиции 0 (просмотров: 4)
Ключ 9: не найден (просмотров: 1)

Записываем результат в красно-чёрное дерево...
Inorder обход дерева (должен совпадать с отсортированным стеком):
42
```

Граничный случай: сортировка возвращает управление сразу (менее двух элементов). Поиск завершается за минимальное число просмотров.

4.4. Тест 4: несуществующий файл

```
$ ./sort missing.txt
Чтение файла 'missing.txt'...
Ошибка: не удалось открыть файл 'missing.txt'
```

Программа корректно обрабатывает ошибку открытия файла и завершает работу с кодом EXIT_FAILURE.

4.5. Тест 5: пустой файл

```
$ ./sort empty.txt
Чтение файла 'empty.txt'...
Ошибка: файл 'empty.txt' пуст или не содержит чисел
```

Программа корректно обнаруживает пустой файл и завершает работу с кодом EXIT_FAILURE.

5. Блок-схемы алгоритма

Блок-схемы всех подпрограмм приведены в Приложении А (стр. 30).

Перечень блок-схем:

1. Главная программа (`main`)
2. Чтение из файла (`read_file`)
3. Цифровая сортировка (`radix_sort`)
4. Сортировка по разряду через стеки-корзины (`counting_sort_by_stack`)
5. Добавление на вершину стека (`stack_push`)
6. Извлечение с вершины стека (`stack_pop`)
7. Поиск методом Бойера-Мура (`boyer_moore_search`)

6. Исходный код программы

Программа состоит из трёх файлов: `stack.h` — объявления структур и всех функций; `stack.c` — реализация стека, алгоритма сортировки, поиска Бойера-Мура и красно-чёрного дерева; `main.c` — точка входа, ВВОД/ВЫВОД.

6.1. Файл `stack.h`

```
1  #ifndef STACK_H
2  #define STACK_H
3
4  /*
5   * Заголовочный файл лабораторной работы 4.
6   *
7   * Определяет:
8   *   - Стек на базе динамического массива (основная структура)
9   *   - Интерфейс цифровой сортировки (LSD через 10 стеков-корзин)
10  *   - Поиск методом Бойера-Мура с возвратом позиции числа
11  *   - Простое красно-чёрное дерево для записи результата
12  *
13  * Реализация в stack.c (все три логических блока в одном файле,
14  * как требовалось в предыдущих лабораторных).
15  */
16
17  #include <stddef.h>
18
19  /* ===== СТЕК НА БАЗЕ ДИНАМИЧЕСКОГО МАССИВА ===== */
20
21  /* Стек LIFO. Дно data[0], вершина data[top].
22   * При переполнении ёмкость удваивается (амортизировано O(1) на push). */
23  typedef struct {
24      int *data;      /* буфер на куче */
25      int top;        /* индекс вершины; -1 стек пуст */
26      int capacity;   /* выделенная ёмкость */
27  } Stack;
28
29  /* Инициализация пустого стека */
30  void stack_init(Stack *s);
31
32  /* Проверка на пустоту */
33  int stack_is_empty(const Stack *s);
34
35  /* Количество элементов */
36  int stack_size(const Stack *s);
37
```

```

38  /* Добавление элемента на вершину. Возвращает 0 при успехе, -1 при ошибке памяти. */
39  int stack_push(Stack *s, int value);
40
41  /* Снятие элемента с вершины. Возвращает 0 при успехе, -1 если пуст. Значение через *out.
       ↪ */
42  int stack_pop(Stack *s, int *out);
43
44  /* Просмотр вершины без удаления. Возвращает 0 при успехе, -1 если пуст. */
45  int stack_peek(const Stack *s, int *out);
46
47  /* Освобождение памяти буфера (саму структуру не free) */
48  void stack_free(Stack *s);
49
50  /* ===== ЦИФРОВАЯ СОРТИРОВКА (LSD) ===== */
51
52  /* Цифровая сортировка стека по возрастанию (от дна к вершине).
53   * Поддерживает отрицательные числа.
54   * После сортировки: data[0] наименьший, data[top] наибольший. */
55  void radix_sort(Stack *s);
56
57  /* ===== ПОИСК МЕТОДОМ БОЙЕРА-МУРА ===== */
58
59  /* Поиск методом Бойера-Мура.
60   * Строит текстовую строку из элементов стека (от дна к вершине) в формате ";число;число
       ↪ ;...;".
61   * Ключ преобразуется в паттерн ";ключ;".
62   * Возвращает позицию числа в отсортированном стеке (0 дно/наименьший, ..., size-1 верш
       ↪ ина)
63   * или -1, если ключ не найден.
64   * В *views количество сравнений символов, выполненных алгоритмом Бойера-Мура. */
65  int boyer_moore_search(const Stack *s, int key, int *views);
66
67  /* ===== КРАСНО-ЧЁРНОЕ ДЕРЕВО ===== */
68
69  typedef enum { BLACK, RED } Color;
70
71  typedef struct RBNode {
72      int value;
73      Color color;
74      struct RBNode *left;
75      struct RBNode *right;
76      struct RBNode *parent;
77  } RBNode;
78
79  typedef struct {
80      RBNode *root;
81  } RBTree;
82

```

```

83  /* Инициализация пустого дерева */
84  void rbtree_init(RBTree *t);
85
86  /* Вставка значения (дубликаты допускаются) */
87  void rbtree_insert(RBTree *t, int value);
88
89  /* Вывод значений дерева в порядке inorder (должен совпадать с отсортированным стеком) */
90  void rbtree_inorder_print(const RBTree *t);
91
92  /* Освобождение всех узлов дерева */
93  void rbtree_free(RBTree *t);
94
95  #endif /* STACK_H */

```

6.2. Файл stack.c

Операции стека

```

1  #include "stack.h"
2
3  #include <stdio.h>
4  #include <stdlib.h>
5  #include <string.h>
6
7  #define BASE 10
8  #define STACK_INIT_CAP 8
9
10 /* ===== РЕАЛИЗАЦИЯ СТЕКА ===== */
11
12 void stack_init(Stack *s)
13 {
14     s->data = NULL;
15     s->top = -1;
16     s->capacity = 0;
17     /* обнуляем структуру стека */
18 }
19
20 int stack_is_empty(const Stack *s)
21 {
22     return s->top == -1; /* проверка пустоты по top */
23 }
24
25 int stack_size(const Stack *s)
26 {
27     return s->top + 1; /* количество элементов */
28 }
29

```



```

30 int stack_push(Stack *s, int value)
31 {
32     if (s->top + 1 >= s->capacity) {
33         int new_cap = (s->capacity == 0) ? STACK_INIT_CAP : s->capacity * 2;
34         int *tmp = (int *)realloc(s->data, new_cap * sizeof(int));
35         if (!tmp) {
36             fprintf(stderr, "Ошибка: не удалось выделить память для стека\n");
37             return -1;
38         }
39         s->data = tmp;
40         s->capacity = new_cap; /* увеличиваем ёмкость */
41     }
42     s->top++;
43     s->data[s->top] = value;
44     return 0; /* элемент добавлен на вершину */
45 }
46
47 int stack_pop(Stack *s, int *out)
48 {
49     if (stack_is_empty(s)) {
50         fprintf(stderr, "Ошибка: стек пуст\n");
51         return -1;
52     }
53     *out = s->data[s->top]; /* берём значение с вершины */
54     s->top--;
55     return 0; /* сняли элемент */
56 }
57
58 int stack_peek(const Stack *s, int *out)
59 {
60     if (stack_is_empty(s)) {
61         fprintf(stderr, "Ошибка: стек пуст\n");
62         return -1;
63     }
64     *out = s->data[s->top]; /* смотрим вершину без удаления */
65     return 0;
66 }
67
68 void stack_free(Stack *s)
69 {
70     free(s->data);
71     s->data = NULL;
72     s->top = -1;
73     s->capacity = 0; /* освобождаем буфер */
74 }
75
76 /* ===== ВСПОМОГАТЕЛЬНЫЕ ДЛЯ СОРТИРОВКИ ===== */
77

```

```

78 static int find_max(const Stack *s)
79 {
80     if (stack_is_empty(s)) return 0;
81     int max_val = s->data[0];
82     for (int i = 1; i <= s->top; i++) {
83         if (s->data[i] > max_val) {
84             max_val = s->data[i]; /* обновляем максимум */
85         }
86     }
87     return max_val; /* максимум определяет число разрядов */
88 }

```

Цифровая сортировка

```

1
2 /* Один проход цифровой сортировки по разряду exp (через стеки-корзины) */
3 static void counting_sort_by_stack(Stack *s, int exp)
4 {
5     if (stack_size(s) <= 1) return;
6
7     Stack buckets[BASE];
8     for (int j = 0; j < BASE; j++) {
9         stack_init(&buckets[j]); /* инициализируем 10 корзин */
10    }
11
12    /* реверс, чтобы обрабатывать от дна (как будто в порядке поступления) */
13    Stack rev;
14    stack_init(&rev);
15    int val;
16    while (!stack_is_empty(s)) {
17        stack_pop(s, &val);
18        stack_push(&rev, val); /* перекладываем в обратном порядке */
19    }
20
21    /* распределение по корзинам */
22    while (!stack_is_empty(&rev)) {
23        stack_pop(&rev, &val);
24        int digit = (val / exp) % BASE;
25        stack_push(&buckets[digit], val); /* кидаем в нужную корзину */
26    }
27    stack_free(&rev); /* rev больше не нужен */
28
29    /* сбор из корзин 0..9 с восстановлением порядка через temp */
30    for (int j = 0; j < BASE; j++) {
31        Stack temp;
32        stack_init(&temp);
33

```

```

34     while (!stack_is_empty(&buckets[j])) {
35         stack_pop(&buckets[j], &val);
36         stack_push(&temp, val); /* реверс содержимого корзины */
37     }
38
39     while (!stack_is_empty(&temp)) {
40         stack_pop(&temp, &val);
41         stack_push(s, val); /* второй реверс восстанавливает порядок */
42     }
43     stack_free(&temp); /* temp для этой корзины готов */
44 }
45
46 for (int j = 0; j < BASE; j++) {
47     stack_free(&buckets[j]); /* освобождаем корзины */
48 }
49 }
50
51 /* сортировка неотрицательных (определяет число проходов по тах) */
52 static void radix_sort_nonneg(Stack *s)
53 {
54     if (stack_size(s) <= 1) return;
55
56     int max_val = find_max(s);
57     for (int exp = 1; max_val / exp > 0; exp *= BASE) {
58         counting_sort_by_stack(s, exp); /* проход по разряду */
59     }
60 }
61
62 /* ===== ОСНОВНАЯ ЦИФРОВАЯ СОРТИРОВКА ===== */
63
64 /* Основная цифровая сортировка стека (с обработкой отрицательных) */
65 void radix_sort(Stack *s)
66 {
67     int n = stack_size(s);
68     if (n <= 1) return;
69
70     Stack neg, pos;
71     stack_init(&neg);
72     stack_init(&pos); /* готовим два вспомогательных стека */
73
74     int val;
75     while (!stack_is_empty(s)) {
76         stack_pop(s, &val);
77         if (val < 0) {
78             stack_push(&neg, -val); /* сохраняем модуль */
79         } else {
80             stack_push(&pos, val);
81         }

```

```

82     }
83
84     radix_sort_nonneg(&neg);
85     radix_sort_nonneg(&pos);  /* сортируем отдельно */
86
87     /* собираем отрицательные (большие модули первыми самые отрицательные внизу) */
88     while (!stack_is_empty(&neg)) {
89         stack_pop(&neg, &val);
90         stack_push(s, -val); /* первое push самое отрицательное в дно */
91     }
92
93     /* реверс положительных, чтобы маленькие шли первыми после отрицательных */
94     Stack pos_rev;
95     stack_init(&pos_rev);
96     while (!stack_is_empty(&pos)) {
97         stack_pop(&pos, &val);
98         stack_push(&pos_rev, val); /* реверс pos */
99     }
100     while (!stack_is_empty(&pos_rev)) {
101         stack_pop(&pos_rev, &val);
102         stack_push(s, val); /* теперь по возрастанию */
103     }
104     stack_free(&pos_rev);
105
106     stack_free(&neg);
107     stack_free(&pos); /* чистим вспомогательные */
108 }

```

Поиск Бойера-Мура

```

1
2  /* ===== ПОИСК БОЙЕРА-МУРА ===== */
3
4  /* Строит текст ";число;число;..." от дна к вершине (для поиска целыми токенами) */
5  static char *build_text_from_stack(const Stack *s)
6  {
7      if (stack_is_empty(s)) {
8          char *empty = (char *)malloc(3);
9          if (empty) strcpy(empty, ";;");
10         return empty;
11     }
12
13     /* Оценим размер буфера */
14     size_t buf_size = (size_t)stack_size(s) * 16 + 4;
15     char *text = (char *)malloc(buf_size);
16     if (!text) return NULL;
17

```

```

18     text[0] = '\0';
19     strcat(text, ";"); /* ведущий разделитель */
20
21     for (int i = 0; i <= s->top; i++) {
22         char numstr[32];
23         snprintf(numstr, sizeof(numstr), "%d", s->data[i]);
24         strcat(text, numstr);
25         strcat(text, ";"); /* каждый элемент + разделитель */
26     }
27     return text; /* текст вида ;val;val;... */
28 }
29
30 /* Преобразует позицию совпадения в тексте в индекс числа в стеке (0 = дно) */
31 static int find_element_index(const Stack *s, int match_start)
32 {
33     if (match_start < 0 || stack_is_empty(s)) return -1;
34
35     int offset = 0; /* позиция открывающего ';' текущего токена */
36
37     for (int i = 0; i <= s->top; i++) {
38         if (match_start == offset) {
39             return i; /* нашли, какой по счёту элемент */
40         }
41
42         char numstr[32];
43         int dlen = snprintf(numstr, sizeof(numstr), "%d", s->data[i]);
44
45         /* Продвигаемся: цифры + замыкающий ';' (он же открывающий для следующего) */
46         offset += dlen + 1; /* двигаем указатель на следующий токен */
47     }
48     return -1;
49 }
50
51 /* Таблица плохого символа (bad character) для алфавита 256 */
52 static void compute_bad_char(const char *pat, int m, int bad[256])
53 {
54     for (int i = 0; i < 256; i++) {
55         bad[i] = -1;
56     }
57     for (int i = 0; i < m; i++) {
58         bad[(unsigned char)pat[i]] = i;
59     }
60 }
61
62 /* Классический Бойер-Мур с эвристикой плохого символа.
63  * Возвращает индекс первого вхождения или -1.
64  * *views увеличивается на каждое сравнение символов. */
65 static int boyer_moore(const char *txt, int n, const char *pat, int m, int *views)

```

```

66 {
67     *views = 0;
68     if (m == 0) return 0;
69     if (n < m) return -1;
70
71     int bad[256];
72     compute_bad_char(pat, m, bad); /* предобработка таблицы плохого символа */
73
74     int shift = 0;
75     while (shift <= n - m) {
76         int j = m - 1;
77
78         /* Сравниваем с конца паттерна */
79         while (j >= 0 && txt[shift + j] == pat[j]) {
80             (*views)++;
81             j--;
82         }
83
84         if (j < 0) {
85             /* Полное совпадение */
86             return shift;
87         } else {
88             (*views)++;
89         }
90
91         /* Сдвиг по плохому символу */
92         int bad_char = (unsigned char)txt[shift + j];
93         int bad_idx = bad[bad_char];
94         int move = (j - bad_idx > 1) ? (j - bad_idx) : 1;
95         shift += move; /* пропускаем по эвристике */
96     }
97     return -1;
98 }
99
100 /* Поиск Бойера-Мура: возвращает позицию числа в стеке + количество просмотров */
101 int boyer_moore_search(const Stack *s, int key, int *views)
102 {
103     char key_str[32];
104     snprintf(key_str, sizeof(key_str), "%d", key);
105
106     /* Оборачиваем ключ в разделители, чтобы искать как целый токен */
107     char pat[40];
108     snprintf(pat, sizeof(pat), ";%s;", key_str); /* паттерн ";75;" */
109
110     char *text = build_text_from_stack(s);
111     if (!text) {
112         *views = 0;
113         return -1;

```

```

114     }
115
116     int n = (int)strlen(text);
117     int m = (int)strlen(pat);
118
119     int char_pos = boyer_moore(text, n, pat, m, views); /* запускаем БМ */
120     int elem_idx = (char_pos >= 0) ? find_element_index(s, char_pos) : -1;
121
122     free(text);
123     return elem_idx; /* возвращаем позицию числа, а не символа */
124 }

```

Красно-чёрное дерево

```

1
2  /* ===== КРАСНО-ЧЁРНОЕ ДЕРЕВО ===== */
3
4  static RBNode *create_node(int value)
5  {
6      RBNode *node = (RBNode *)malloc(sizeof(RBNode));
7      if (!node) {
8          fprintf(stderr, "Ошибка: malloc для узла RBT\n");
9          exit(EXIT_FAILURE);
10     }
11     node->value = value;
12     node->color = RED;
13     node->left = NULL;
14     node->right = NULL;
15     node->parent = NULL;
16     return node; /* новый красный узел */
17 }
18
19 static void left_rotate(RBTree *t, RBNode *x)
20 {
21     RBNode *y = x->right;
22     x->right = y->left;
23     if (y->left != NULL) {
24         y->left->parent = x;
25     }
26     y->parent = x->parent;
27     if (x->parent == NULL) {
28         t->root = y;
29     } else if (x == x->parent->left) {
30         x->parent->left = y;
31     } else {
32         x->parent->right = y;
33     }

```

```

34     y->left = x;
35     x->parent = y;  /* левый поворот выполнен */
36 }  /* левый поворот */
37
38 static void right_rotate(RBTree *t, RBNode *y)
39 {
40     RBNode *x = y->left;
41     y->left = x->right;
42     if (x->right != NULL) {
43         x->right->parent = y;
44     }
45     x->parent = y->parent;
46     if (y->parent == NULL) {
47         t->root = x;
48     } else if (y == y->parent->right) {
49         y->parent->right = x;
50     } else {
51         y->parent->left = x;
52     }
53     x->right = y;
54     y->parent = x;  /* структура связей обновлена */
55 }  /* правый поворот */
56
57 static void insert_fixup(RBTree *t, RBNode *z)
58 {
59     while (z->parent != NULL && z->parent->color == RED) {
60         RBNode *gp = z->parent->parent;
61         if (z->parent == gp->left) {
62             RBNode *y = gp->right;  /* uncle */
63             if (y != NULL && y->color == RED) {
64                 /* случай 1: дядя красный перекрашиваем */
65                 z->parent->color = BLACK;
66                 y->color = BLACK;
67                 gp->color = RED;
68                 z = gp;
69             } else {
70                 if (z == z->parent->right) {
71                     /* случай 2: левый поворот */
72                     z = z->parent;
73                     left_rotate(t, z);
74                 }
75                 /* случай 3: правый поворот + перекраска */
76                 z->parent->color = BLACK;
77                 gp->color = RED;
78                 right_rotate(t, gp);
79             }
80         } else {
81             RBNode *y = gp->left;

```



```

82         if (y != NULL && y->color == RED) {
83             z->parent->color = BLACK;
84             y->color = BLACK;
85             gp->color = RED;
86             z = gp;
87         } else {
88             if (z == z->parent->left) {
89                 z = z->parent;
90                 right_rotate(t, z);
91             }
92             z->parent->color = BLACK;
93             gp->color = RED;
94             left_rotate(t, gp);
95         }
96     }
97 }
98 t->root->color = BLACK; /* корень всегда чёрный после фикса */
99 }
100
101 void rbtree_init(RBTree *t)
102 {
103     t->root = NULL; /* пустое дерево */
104 }
105
106 void rbtree_insert(RBTree *t, int value) /* вставка с балансировкой */
107 {
108     RBNode *z = create_node(value);
109     RBNode *y = NULL;
110     RBNode *x = t->root;
111
112     while (x != NULL) {
113         y = x;
114         if (z->value < x->value) {
115             x = x->left;
116         } else {
117             x = x->right;
118         }
119     } /* нашли место для вставки */
120
121     z->parent = y;
122     if (y == NULL) {
123         t->root = z;
124     } else if (z->value < y->value) {
125         y->left = z;
126     } else {
127         y->right = z;
128     }
129 }

```

```

130     insert_fixup(t, z); /* восстанавливаем свойства RBT */
131 }
132
133 static void inorder_helper(const RBNode *node)
134 {
135     if (node == NULL) return;
136     inorder_helper(node->left);
137     printf("%d ", node->value);
138     inorder_helper(node->right);
139 }
140
141 void rbtree_inorder_print(const RBTree *t)
142 {
143     inorder_helper(t->root);
144     printf("\n");
145 }
146
147 static void free_helper(RBNode *node)
148 {
149     if (node == NULL) return;
150     free_helper(node->left);
151     free_helper(node->right);
152     free(node);
153 }
154
155 void rbtree_free(RBTree *t)
156 {
157     free_helper(t->root);
158     t->root = NULL;

```

6.3. Файл main.c

```

1 #include "stack.h"
2
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <string.h>
6
7 /* чтение файла в стек через push */
8 static int read_file(const char *filename, Stack *s)
9 {
10     FILE *fp = fopen(filename, "r");
11     if (!fp) {
12         fprintf(stderr, "Ошибка: не удалось открыть файл '%s'\n", filename);
13         return -1;
14     }
15

```

```

16     int tmp;
17     int count = 0;
18     while (fscanf(fp, "%d", &tmp) == 1) {
19         if (stack_push(s, tmp) != 0) {
20             stack_free(s);
21             fclose(fp);
22             return -1;
23         }
24         count++; /* считаем прочитанные */
25     }
26
27     fclose(fp);
28
29     if (count == 0) {
30         fprintf(stderr, "Ошибка: файл '%s' пуст или не содержит чисел\n", filename);
31         return -1;
32     }
33     return count; /* вернули размер */
34 }
35
36 /* печать стека от дна к вершине */
37 static void stack_print(const Stack *s)
38 {
39     if (stack_is_empty(s)) {
40         printf("\n");
41         return;
42     }
43     for (int i = 0; i <= s->top; i++) {
44         printf("%d", s->data[i]);
45         if (i < s->top) {
46             printf(" ");
47         }
48     }
49     printf("\n"); /* вывод от дна к вершине */
50 }
51
52 /* демонстрация поиска Бойера-Мура по ключу */
53 static void search_demo(const Stack *s, int key)
54 {
55     int views;
56     int pos = boyer_moore_search(s, key, &views);
57
58     char pat[32];
59     snprintf(pat, sizeof(pat), "%d", key); /* ключ как строка для вывода */
60
61     if (pos != -1) {
62         printf("Ключ %s: НАЙДЕН на позиции %d (просмотров: %d)\n",
63             pat, pos, views);

```

```

64     } else {
65         printf("Ключ %s: не найден (просмотров: %d)\n", pat, views);
66     }
67 } /* конец демо поиска */
68
69 /* запись результата в RBT и вывод inorder (проверка сортировки) */
70 static void rbt_demo(Stack *s)
71 {
72     RBTtree tree;
73     rbtree_init(&tree);
74
75     printf("\nЗаписываем результат в красно-чёрное дерево...\n");
76
77     for (int i = 0; i <= s->top; i++) {
78         rbtree_insert(&tree, s->data[i]); /* вставляем каждый элемент */
79     }
80
81     printf("Inorder обход дерева (должен совпадать с отсортированным стеком):\n");
82     rbtree_inorder_print(&tree);
83
84     rbtree_free(&tree); /* чистим дерево */
85 } /* конец демонстрации RBT */
86
87 int main(int argc, char *argv[])
88 {
89     if (argc < 2) {
90         fprintf(stderr, "Использование: %s <файл> [ключ поиска ...]\n", argv[0]);
91         return EXIT_FAILURE;
92     }
93     const char *filename = argv[1];
94
95     /*
96      * Общий поток программы (согласно заданию лабораторной):
97      * 1. Прочитать числа из файла поместить в стек (push, LIFO).
98      * 2. Вывести исходное содержимое.
99      * 3. Выполнить цифровую (radix) сортировку стека.
100     * 4. Вывести отсортированный стек (дно вершина = по возрастанию).
101     * 5. Продемонстрировать поиск методом Бойера-Мура по ключам.
102     * 6. Записать отсортированный результат в красно-чёрное дерево
103     *    и показать, что inorder-обход даёт тот же порядок.
104     * 7. Освободить память.
105     */
106
107     printf("\nЧтение файла '%s'...\n", filename);
108
109     Stack s;
110     stack_init(&s); /* инициализируем стек */
111

```

```

112     int n = read_file(filename, &s);
113     if (n < 0) {
114         stack_free(&s);
115         return EXIT_FAILURE;
116     }
117
118     /* Склонение слова "число" */
119     const char *form = (n % 100 >= 11 && n % 100 <= 19) ? "чисел"
120         : (n % 10 == 1) ? "число"
121         : (n % 10 >= 2 && n % 10 <= 4) ? "числа"
122         : "чисел";
123     printf("Прочитано %d %s.\n", n, form);
124
125     printf("\nИсходный стек (от дна к вершине):\n");
126     stack_print(&s); /* вывод до сортировки */
127
128     printf("\nВыполняем цифровую сортировку стека...\n");
129     radix_sort(&s); /* сортируем */
130
131     printf("Отсортированный стек (от дна к вершине):\n");
132     stack_print(&s);
133
134     printf("\n Поиск методом Бойера-Мура \n");
135     if (argc > 2) {
136         for (int i = 2; i < argc; i++) {
137             int key = atoi(argv[i]);
138             search_demo(&s, key); /* поиск по аргументам */
139         }
140     } else {
141         printf("Введите ключ для поиска: ");
142         int key;
143         if (scanf("%d", &key) == 1) {
144             search_demo(&s, key);
145         } else {
146             fprintf(stderr, "Ошибка: ключ не прочитан\n");
147         }
148     }
149     printf("\n");
150
151     rbt_demo(&s); /* записываем результат в дерево */
152
153     stack_free(&s); /* финальное освобождение */
154     return EXIT_SUCCESS;
155 }

```

Заключение

В ходе выполнения лабораторной работы была разработана программа на языке C, реализующая цифровую сортировку LSD целых чисел (включая отрицательные), считываемых из файла, с использованием **стека на базе динамического массива** в качестве основной и вспомогательной структуры данных, а также **поиск методом Бойера-Мура** в отсортированном стеке и **запись результата в красно-чёрное дерево**.

Программа организована в три файла: `stack.h` объявляет структуры и интерфейс; `stack.c` реализует стек, сортировку, поиск и дерево; `main.c` содержит точку входа и ввод/вывод.

Поддержка отрицательных чисел реализована через разделение стека на два вспомогательных (`neg` и `pos`), их независимую сортировку и сборку результата с учётом LIFO-порядка.

Поиск методом Бойера-Мура выполняется на строковой сериализации отсортированного стека вида `;val1;val2;...;valN;`, что гарантирует поиск целых числовых токенов.

Результаты тестирования подтверждают корректность сортировки для всех рассмотренных случаев — стандартного, с отрицательными числами, с одним элементом — а также правильную обработку ошибочных ситуаций.

Приложение А. Блок-схемы алгоритма

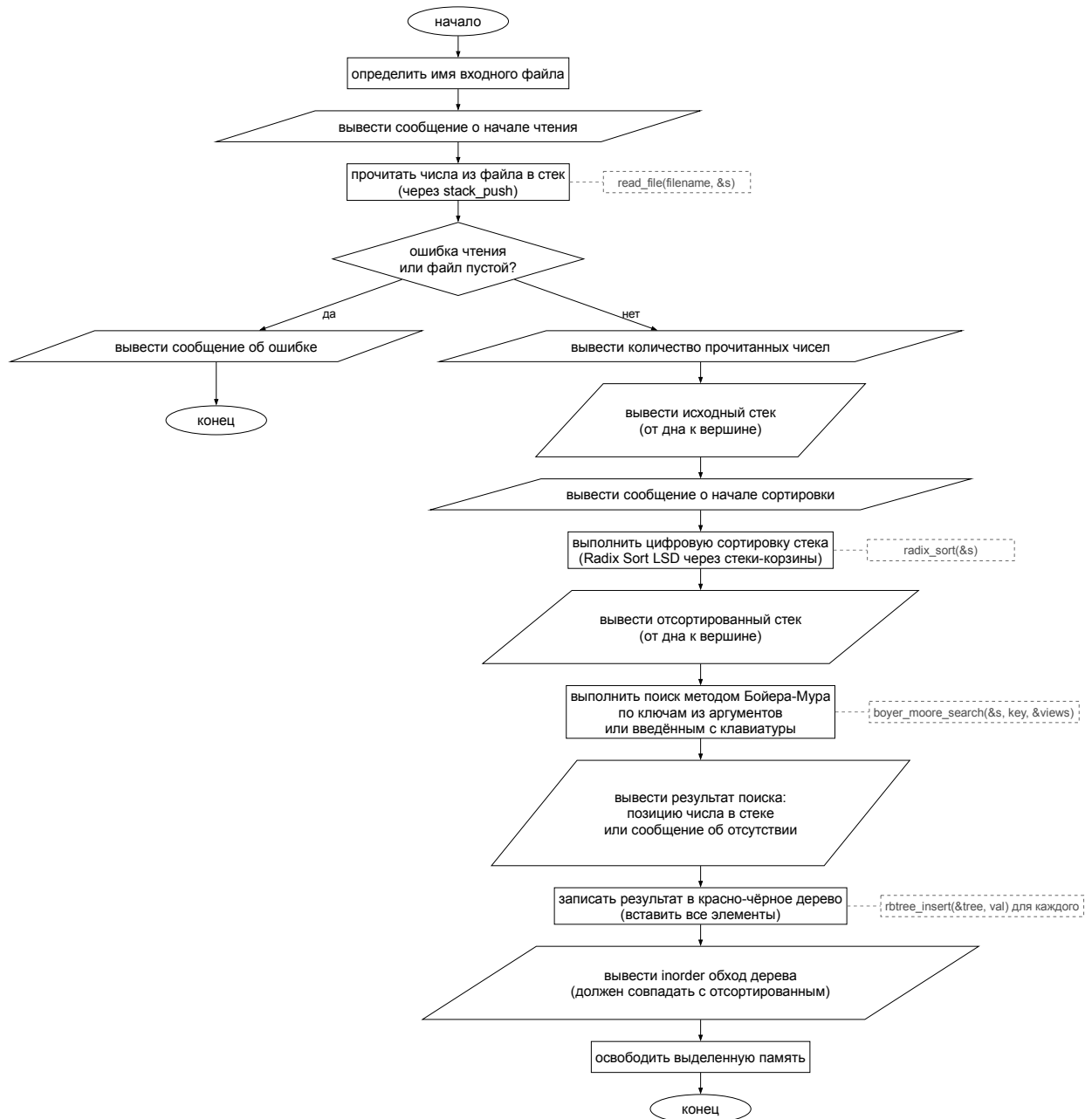


Рис. 1 – Блок-схема функции main

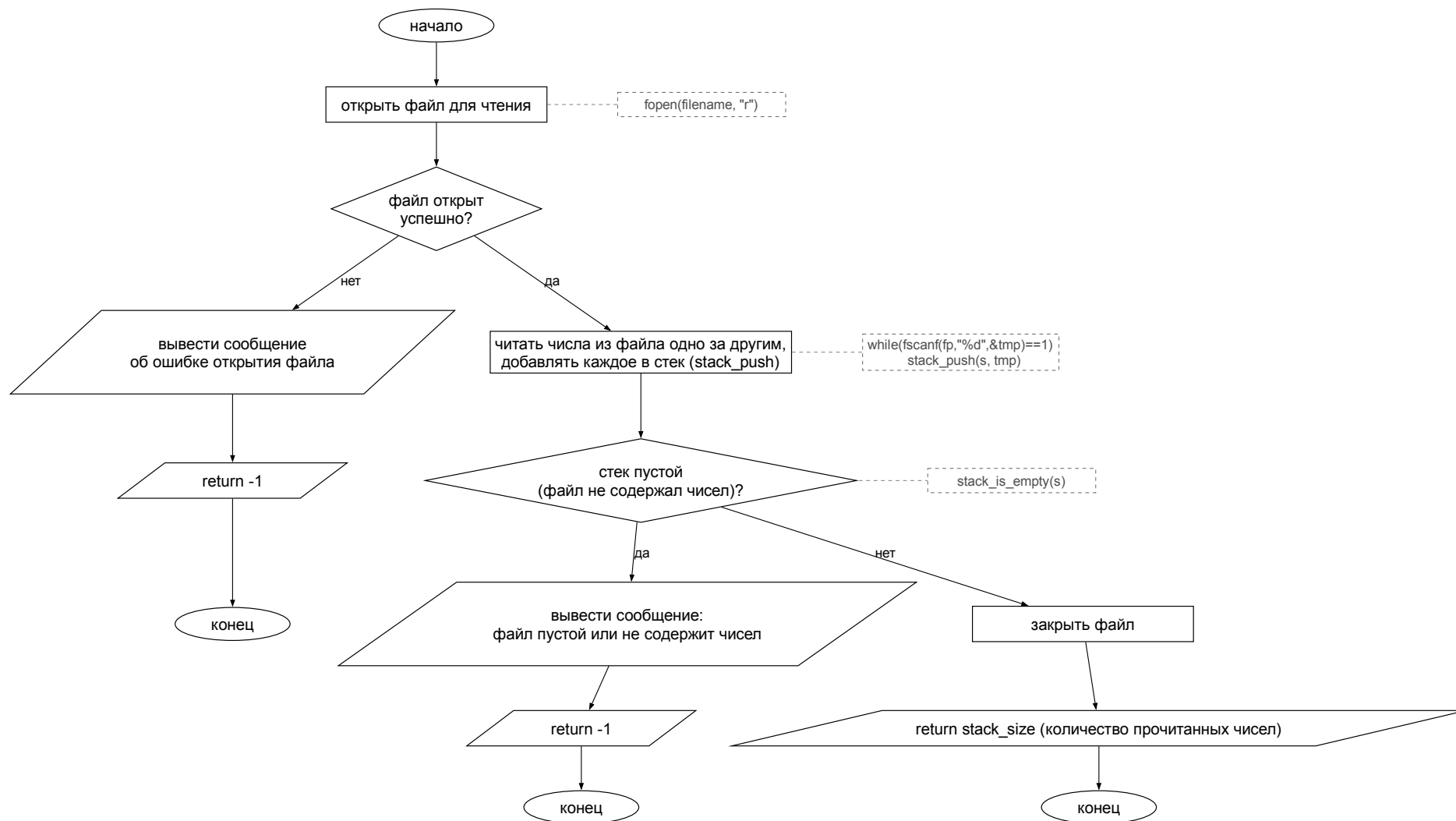


Рис. 2 – Блок-схема чтения файла

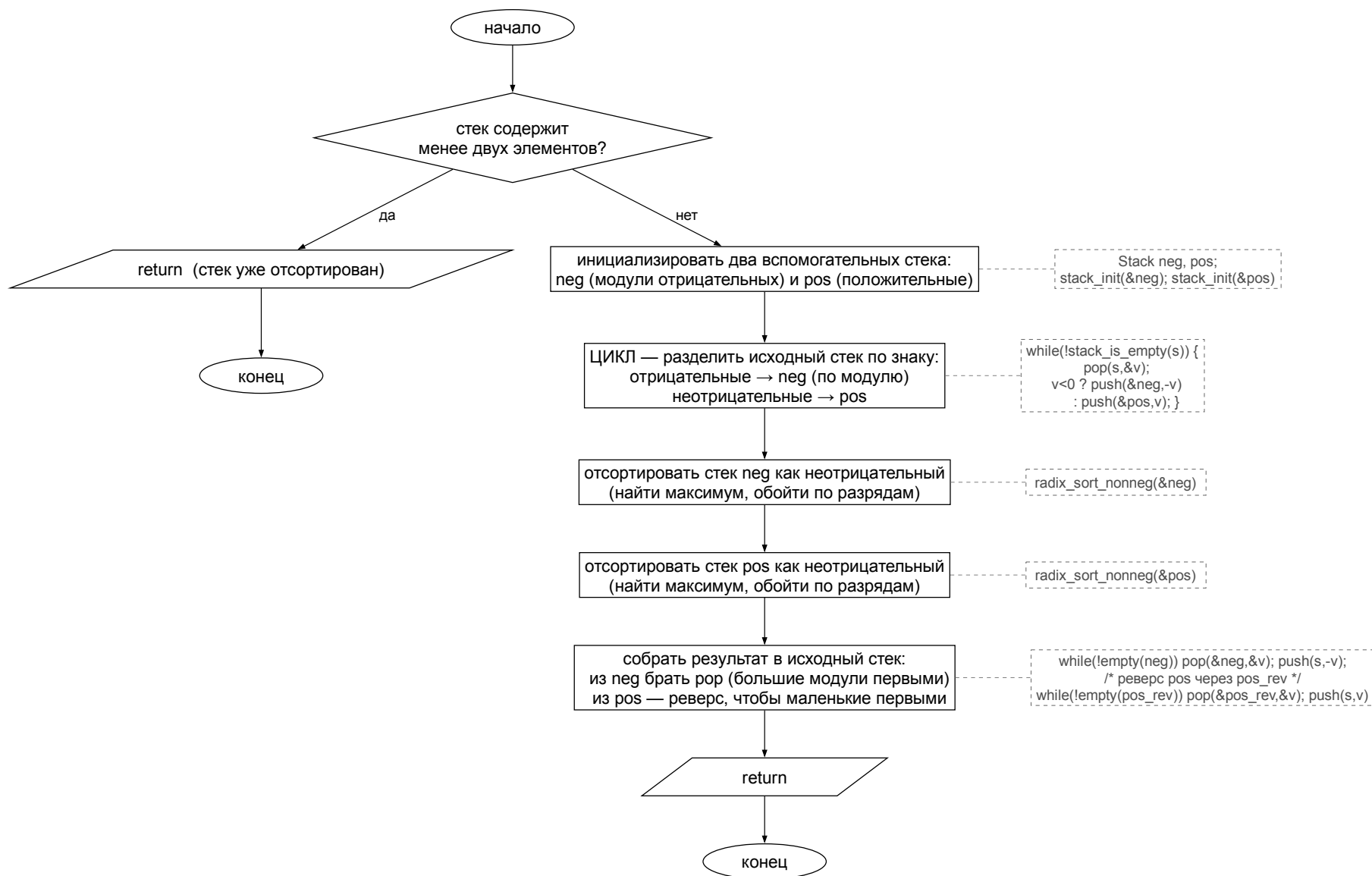


Рис. 3 – Блок-схема radix_sort

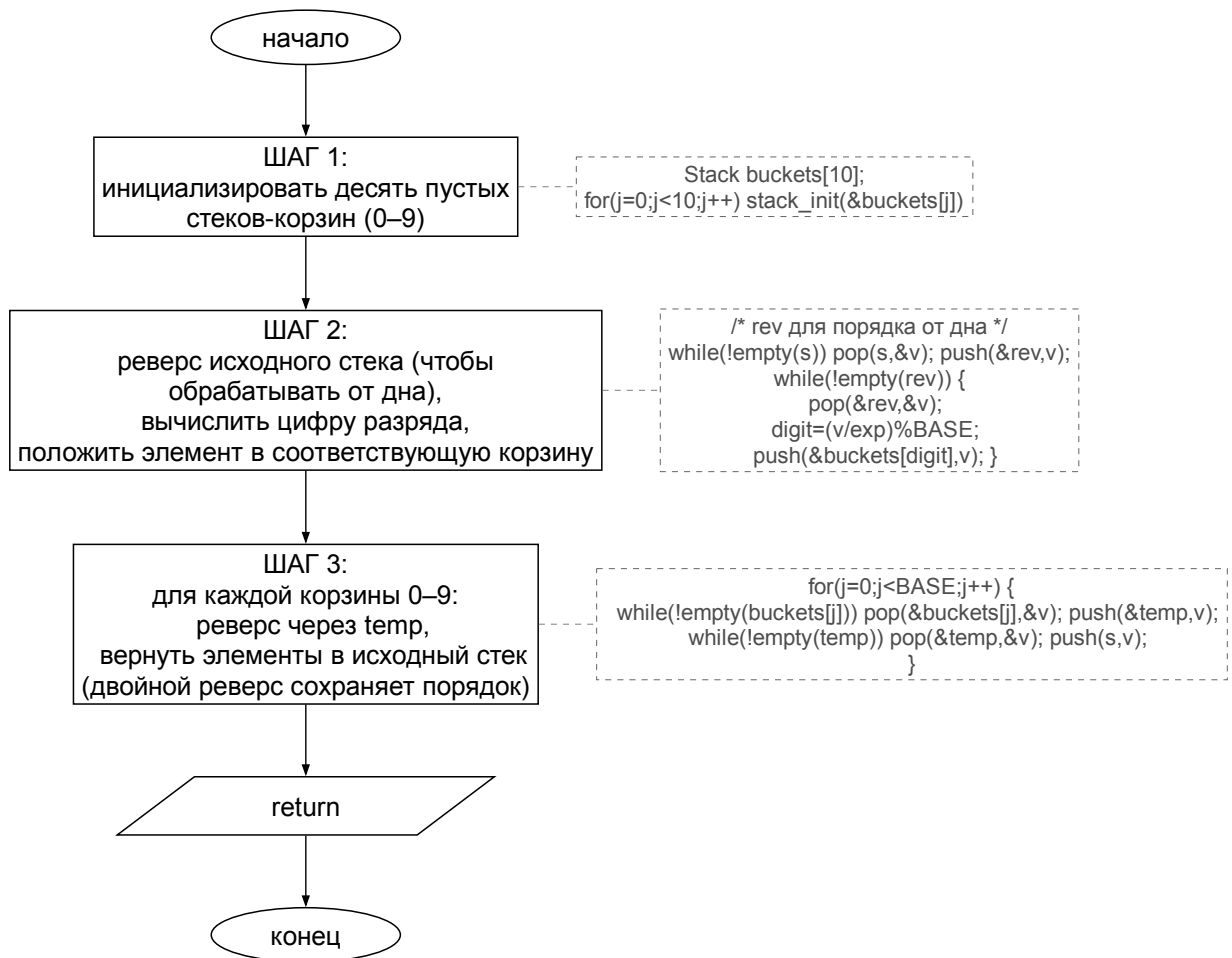


Рис. 4 – Блок-схема одного прохода counting_sort_by_stack

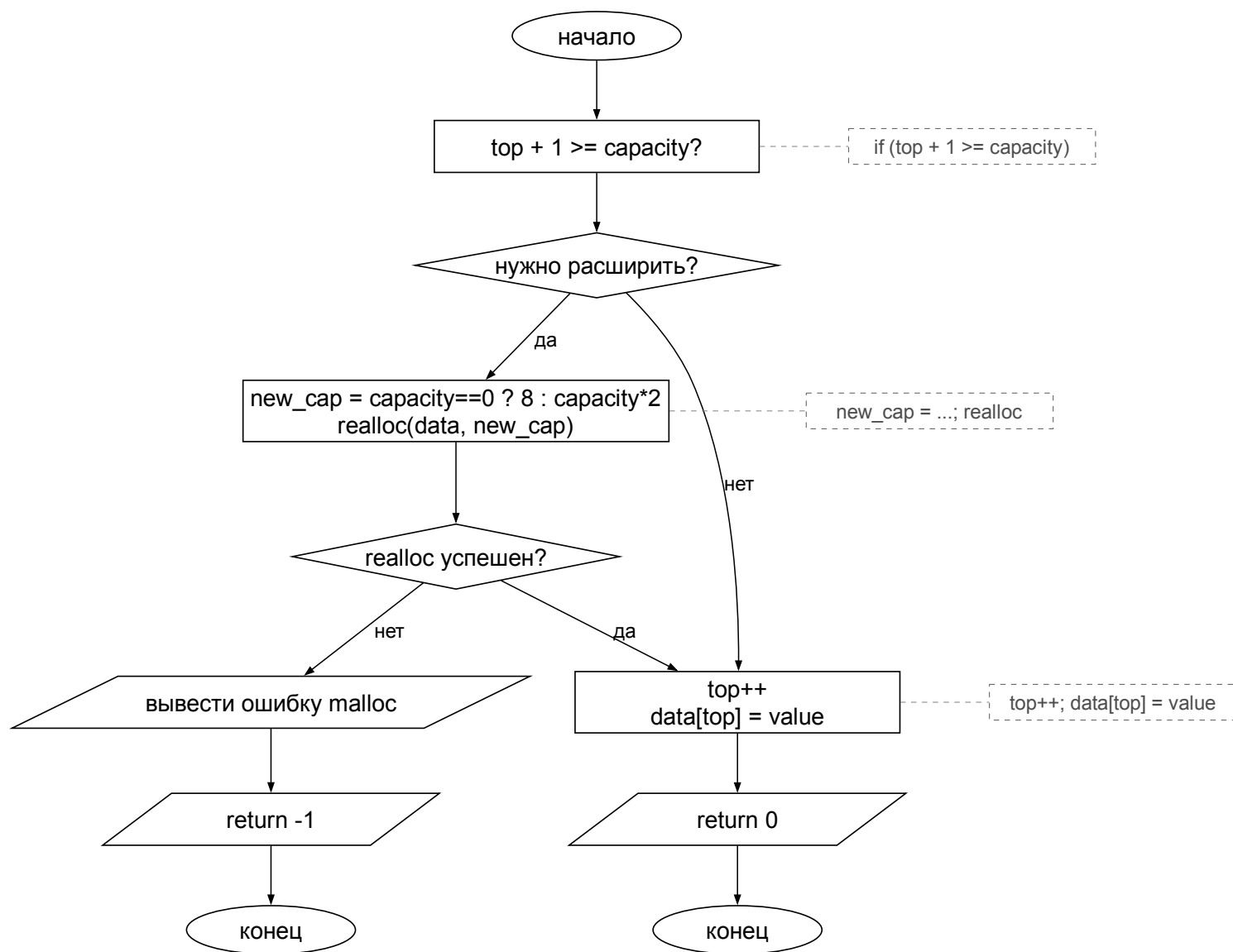


Рис. 5 – Блок-схема stack_push

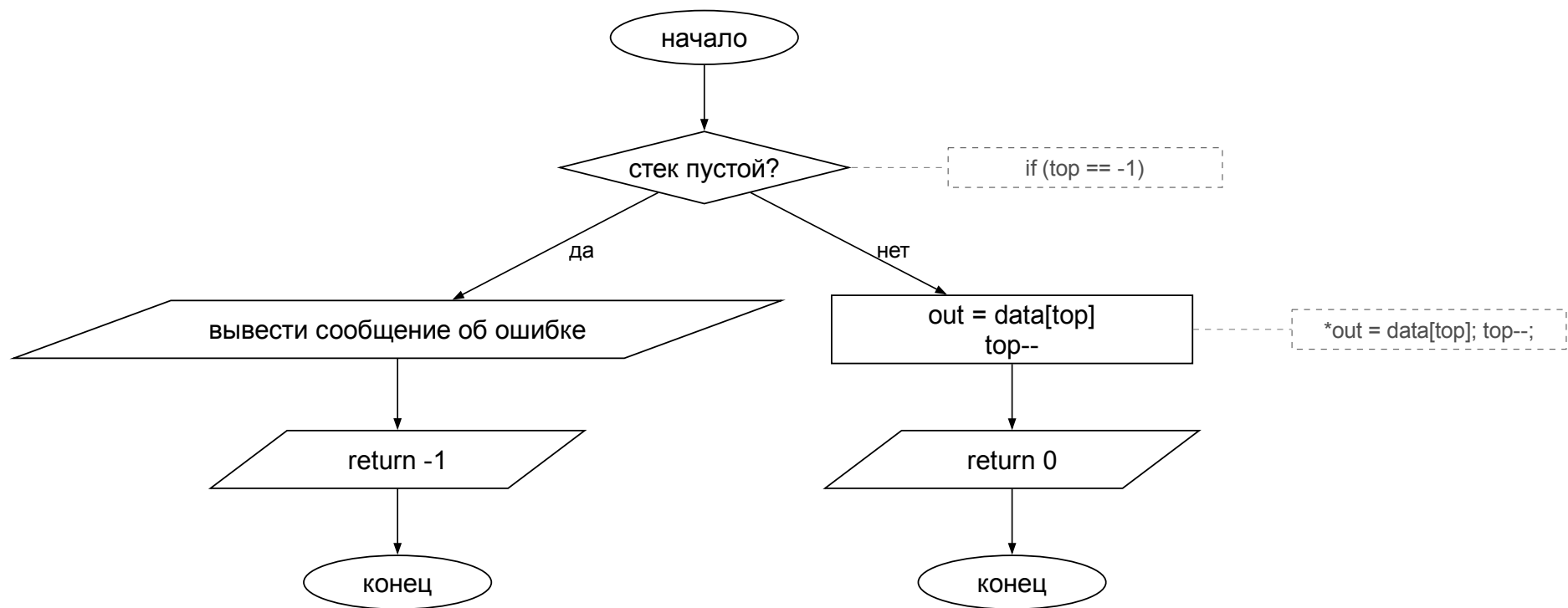


Рис. 6 – Блок-схема `stack_pop`

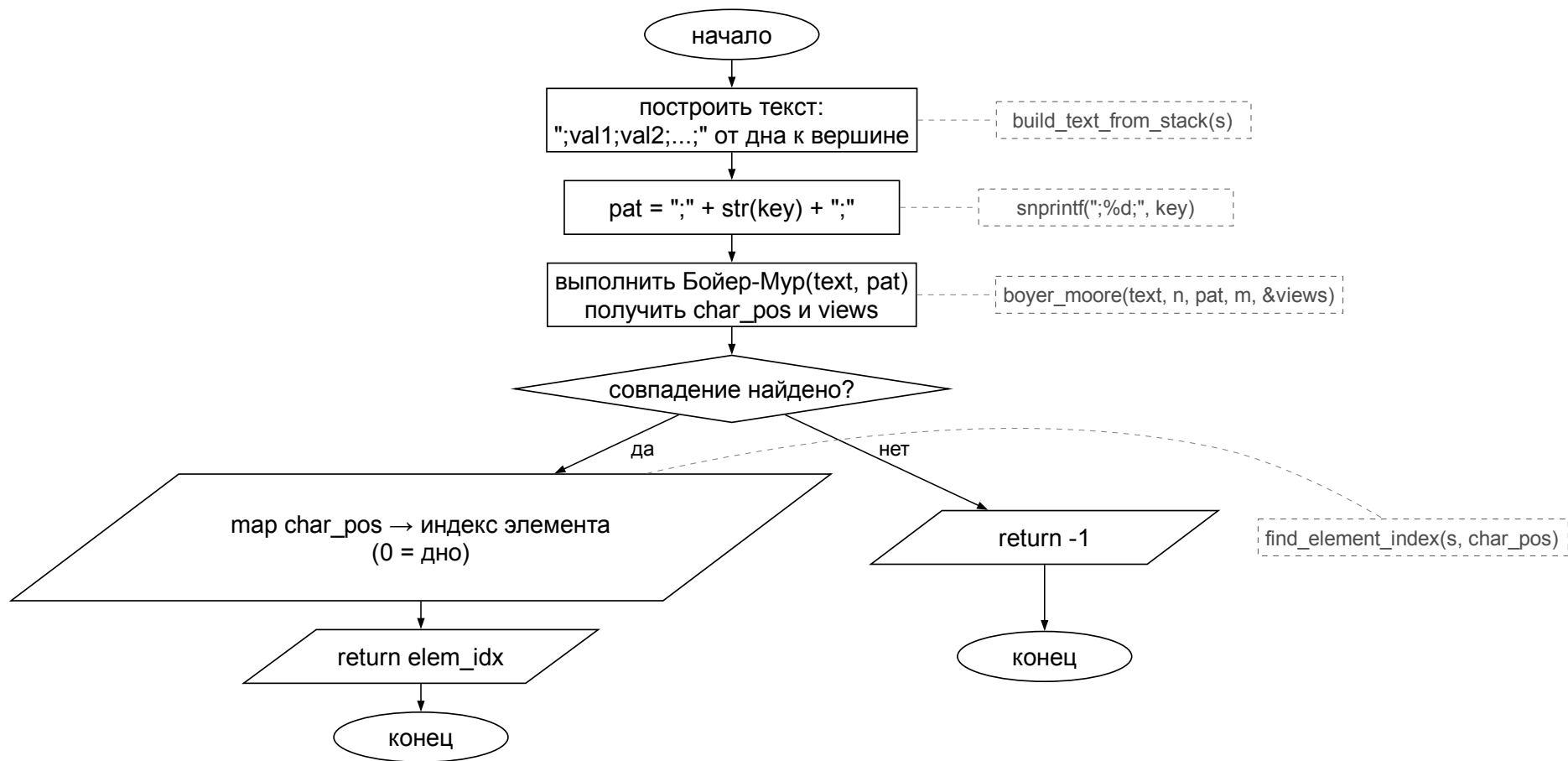


Рис. 7 – Блок-схема boyer_moore_search